

BMDS2114 Machine Learning — Reinforcement Learning Topic Proposal

Domain: User Engagement Optimization

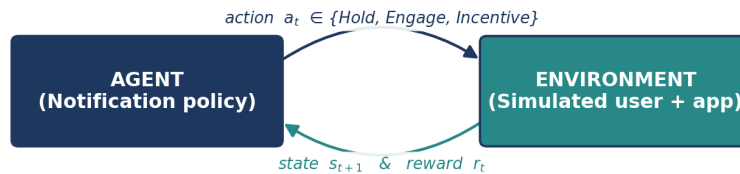
Context-Aware Notification Engine: A Reinforcement Learning Approach to Fatigue-Aware Push-Notification Pacing for User-Engagement Optimization

Topic Proposal

1. Introduction

1.1 Overview of the proposed system

The proposed system, the **Context-Aware Notification Engine (CANE)**, is a platform-side decision agent that governs the push-notification channel of a mobile application (in the style of a social, e-commerce, or content app). At each decision point the agent observes the user's current context and chooses one of three actions — send an *Engagement Nudge*, send an *Incentive Nudge*, or *Hold* (stay silent) — so as to sustain long-term engagement rather than maximise the response to any single message. Crucially, the decision is purely a **communication-timing and message-type** problem at the platform layer: the engine never authors or reasons about the subject-matter of the content, only about whether and when an interruption is worthwhile given the user's evolving state. This keeps the work squarely within the human–computer-interaction / platform-mechanics scope of the User Engagement Optimization domain.



At each timestep the agent observes user context, chooses one action, and receives a reward reflecting clicks, sending cost, fatigue and churn.

Figure 1.0: Reinforcement-learning interaction loop for the Context-Aware Notification Engine.

1.2 Why optimising user engagement matters

- **Retention and revenue:** push notifications are a primary lever for bringing users back into an app, and notification quality has a direct, measured effect on daily-active-users and retention in production systems (Yuan et al., 2022; Cai et al., 2023).
- **Notification fatigue:** poorly timed or excessive notifications annoy users and cause them to ignore, mute, or disable notifications entirely — a largely irreversible loss of the channel (Cai et al., 2023).
- **Trust and user experience:** the value of a notification depends on context and on the history of prior notifications; sending at the wrong moment erodes trust, whereas well-timed relevance preserves it (Ho et al., 2018; Liao et al., 2020).

Engagement is therefore not about sending *more* notifications, but about sending the *right* notification at the *right* moment — and, just as often, about choosing to stay silent.

1.3 How Reinforcement Learning helps

Deciding whether to notify is a **sequential decision-making problem under uncertainty** (Sutton & Barto, 2018): each notification changes the user's fatigue and trust, which in turn changes how the next notification will be received. A myopic, response-prediction model optimises only the immediate click and will tend to over-send, because the negative effects (fatigue, opt-out) are deferred to the future (Cai et al., 2023). Reinforcement Learning (RL) instead learns a *policy* that maximises long-term cumulative engagement, explicitly trading a click now against the risk of fatigue-driven churn later. Because the agent learns from per-user feedback, it adapts its pacing to each user rather than applying one global schedule (Yuan et al., 2022).

1.4 Related work

The table below summarises the most relevant recent studies and their relation to CANE.

Study	Method / focus	Relevance to CANE
Ho et al. (2018), UbiComp	RL agent decides send-vs-stay-silent from sensed user context; trained against a simulated user behaviour + response model.	Direct precedent for an RL send/hold timing agent and for a simulated-user environment.
Tomkins et al. (2021), Machine Learning	Practical Thompson sampling (IntelligentPooling) for mobile-health notification decisions, pooling data across users for sample efficiency.	Justifies a contextual-bandit baseline for notification decisions and its sample-efficiency advantage.
Cai et al. (2023), WWW	RL that directly optimises long-term user retention at billion-user scale, rather than the immediate interaction.	Supports the long-horizon MDP framing and the churn-dominated reward ordering, with opt-out as the terminal event.
Prabhakar et al. (2022), KDD	Offline RL with a Double-DQN / Conservative-Q method; multi-objective notification optimisation.	Basis for our multi-objective scalarised reward and for Double DQN as an algorithm.
Yuan et al. (2022), CIKM	Offline RL for mobile notifications; argues impact must be attributed to a sequence, not a single push.	Confirms the sequential, history-dependent framing and the need for state memory.
Liao et al. (2020), IMWUT	Online RL deployed in the HeartSteps V2 study, deciding several times a day whether to deliver a context-tailored suggestion.	Confirms whether-and-when-to-notify is learnable in deployment, with context and accumulated burden as key signals.

Research gap

Existing systems tend to be either (i) myopic response-prediction or volume-control models that optimise the immediate open, or (ii) bandit formulations that treat each notification independently and so cannot reason about how accumulated fatigue compounds across many steps. Production RL studies that do model the sequence are almost exclusively *offline* and report a single deployed method. Few course-scale studies place a fatigue-aware reward inside a fully interactive environment and **compare a progression of modern RL algorithms (bandit → value-based → improved value-based → policy-based) on the identical task, with Q-Learning deliberately excluded**. CANE targets exactly this gap.

2. Problem Statement

Conventional notification systems are **static and context-blind**: they fire on fixed schedules (e.g., a daily 6:00 p.m. push to every user) regardless of whether the user is asleep, at work, already active in the app,

or already saturated with recent alerts. This produces two failure modes — **(1) wasted communication**, where notifications are sent at moments the user will ignore, and **(2) fatigue-driven attrition**, where cumulative over-messaging drives the user to mute or uninstall. Rule-based heuristics cannot capture the sequential, compounding nature of attention: a notification's value depends on the history of prior notifications and the user's current state (Yuan et al., 2022; Cai et al., 2023). The agent must learn an adaptive pacing policy that maximises long-term engagement while explicitly penalising fatigue.

2.1 What the RL agent must learn

The agent must learn, per user, the optimal action at each timestep — *Hold*, *Engagement Nudge*, or *Incentive Nudge* — conditioned on time of day, day of week, current in-app activity, accumulated notification fatigue, and recency of the last notification, so as to maximise long-term engagement (clicks and sustained sessions) while avoiding fatigue-driven opt-out. It must learn this purely from interaction with a simulated user population and generalise across users with different daily routines without per-user retraining.

2.2 Research questions

RQ1. Can reinforcement learning improve long-term user engagement through fatigue-aware notification pacing, relative to fixed-schedule and random baselines?

RQ2. Among LinUCB, DQN, Double DQN, and PPO, which achieves the best engagement–fatigue trade-off, and does the bandit baseline measurably under-perform the full-MDP methods?

RQ3. Does a single learned policy adapt its pacing to distinct user archetypes (e.g., daytime vs. night-active routines) without per-user retraining?

3. Objectives related to the RL framework

- To design an RL agent that learns a notification-pacing policy maximising long-term engagement (clicks, sustained sessions, retention) rather than the immediate response to a single push.
- To build a simulated user-population environment with five hidden behavioural archetypes (a 9-to-5 Office Worker, a Night-Owl Student, a Night-Shift Worker, a Normal Student, and a flexible Housewife), each with a different time-dependent click propensity and fatigue tolerance, so the agent must infer responsiveness implicitly from reward alone, with no explicit archetype label in the state.
- To design an explainable multi-objective reward (click reward – sending cost – fatigue penalty – churn penalty, plus a retention-streak shaping term) whose magnitudes are justified by their relative ordering and by the literature, not chosen arbitrarily.
- To implement, compare, and justify four techniques — LinUCB, DQN, Double DQN, and PPO (**Q-Learning excluded; SARSA deliberately avoided**) — against two non-adaptive baselines: a fixed daily-6 p.m. schedule and a random sender.
- To evaluate primarily on cumulative reward, click-through rate, and retention (opt-out) rate, with supporting metrics (notification volume, convergence speed, and the per-hour Hold/Send decision distribution), and to verify the learned policy generalises across all five archetypes.

4. Reinforcement Learning Components

The table summarises the five core RL elements; detailed definitions, formulas, and references follow in sub-sections (a)–(g).

Component	Description (this topic)	Reference / basis
Agent	Notification policy that observes user context and selects one of {Hold, Engagement, Incentive} to maximise discounted engagement return.	Ho et al. (2018); Cai et al. (2023)
Environment	Simulated user population (5 hidden archetypes) + app, with a fatigue dynamics model and a click/churn response model.	Ho et al. (2018); Liao et al. (2020)
State	$s_t = [h_t, d_t, A_t, F_t, \rho_t]$: hour, day, in-app activity, fatigue, recency of last notification.	Yuan et al. (2022); Cai et al. (2023)
Action	$a_t \in \{0 \text{ Hold}, 1 \text{ Engagement Nudge}, 2 \text{ Incentive Nudge}\}$.	Tomkins et al. (2021); Ho et al. (2018)
Reward	Click reward – send cost – fatigue penalty – churn penalty + retention-streak shaping.	Hayes et al. (2022); Prabhakar et al. (2022); Sutton & Barto (2018)

a. Agent

The agent is the platform's notification scheduler. At each timestep it observes the state vector and outputs a single discrete action. A single shared-parameter policy is used for the whole population; the state vector provides the personalisation signal, following the per-user RL framing of Ho et al. (2018). The agent's goal is to maximise the expected discounted engagement return (defined in sub-section e).

b. Environment

The environment is a custom, OpenAI-Gym-style simulator implemented in Python. It is **not a static dataset**: RL requires an interactive environment that responds to actions and returns feedback, so user behaviour is **generated by probabilistic models whose parameters are calibrated from the notification literature** rather than sampled from a fixed table (Ho et al., 2018; Liao et al., 2020). At each step the environment (i) decides whether the user clicks, (ii) updates fatigue and recency, and (iii) decides whether the user opts out. To make personalisation visible, five archetypes are defined, each with a different time-of-day responsiveness; each archetype is **hidden** from the agent, so the agent must infer it from reward feedback (Figure 1.3).

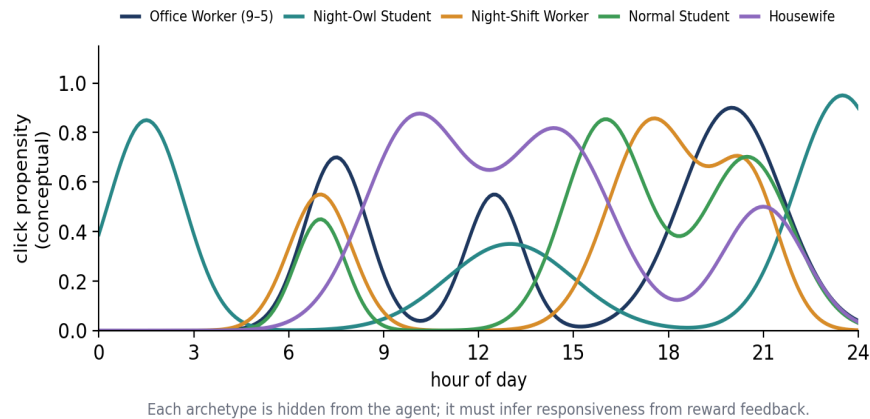


Figure 1.3: Five hidden user archetypes with different time-of-day responsiveness (environment design).

The five archetypes are differentiated by when they are receptive, so that a single policy must learn genuinely different pacing for each:

- **Office Worker (9–5):** largely unresponsive during work hours; receptive at the morning commute, around lunch, and in the evening.
- **Night-Owl Student:** peaks late at night (roughly 22:00–02:00) and is quiet through the morning.
- **Night-Shift Worker:** a phase-inverted routine — asleep through much of the daytime, receptive in the late afternoon before a shift and in the early morning after it.
- **Normal Student:** in class during the day; receptive after school and during evening study breaks, and asleep by around midnight.
- **Housewife:** the most time-flexible — high daytime availability in the mid-morning and early afternoon, with a dip around family / dinner hours, which clearly separates it from the work-bound Office Worker.

Click / response model (the data generator)

When the agent sends action $a \in \mathcal{A}$ the probability that the user clicks and starts a session is modelled as a calibrated, fatigue-attenuated propensity, in line with the calibrated user-response models used in RL notification studies (Ho et al., 2018):

$$P(\text{click} \mid s_t, a_t) = \text{clip}(p_0(\text{arch}, h_t, d_t) \cdot (1 - \mu \cdot F_t) \cdot w(a_t), 0, 1)$$

$p_0(\text{arch}, h_t, d_t)$ is the archetype's baseline responsiveness at the current time, calibrated from reported best-send-time effects (Ho et al., 2018; Liao et al., 2020); $(1 - \mu \cdot F_t)$ is the fatigue attenuation (more accumulated notifications lower the open probability — the burden/annoyance attenuation reported in deployed intervention RL (Liao et al., 2020)); and $w(a_t)$ is the relevance weight of the message type ($w(\text{Hold})=0$). No numeric value is invented arbitrarily: each factor is grounded in a cited mechanism, and the absolute scale is fixed during calibration so that baseline open rates match published ranges.

Fatigue dynamics

Rather than letting the agent choose a fatigue level, the environment maintains it as an internal accumulation–recovery process. Each send raises fatigue; each Hold lets it decay, capturing the burden accumulation–recovery dynamic emphasised in the digital-intervention literature (Liao et al., 2020; Trella et al., 2022):

$$F_{t+1} = \text{clip}(\lambda \cdot F_t + \kappa \cdot I\{a_t \neq 0\}, 0, 1), \quad 0 < \lambda < 1$$

The relative magnitudes are justified rather than free: the recovery factor $\lambda = 0.90$ keeps fatigue slow to forget, while the per-send increment $\kappa = 0.15$ is moderate, so that repeated sends compound into high fatigue faster than Holds can dissipate it — exactly the asymmetry that makes over-messaging dangerous. Fatigue then feeds back into both the click model and the churn model (Figure 1.2).

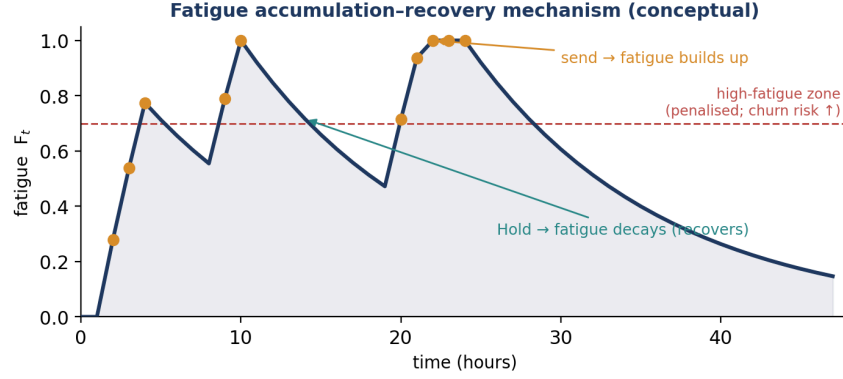


Figure 1.2: Fatigue accumulation–recovery mechanism — sends build fatigue, Holds let it recover (conceptual).

Churn model

Sustained high fatigue raises the probability of opt-out/uninstall, a terminal event that ends all future reward (Cai et al., 2022):

$$P(\text{opt-out} \mid F_t) = \sigma(\gamma_0 + \gamma_1 \cdot F_t), \quad \text{triggered only while } F_t \text{ exceeds a fatigue threshold}$$

where σ is the logistic function. Because opt-out removes the channel permanently, its penalty must dominate the reward (sub-section e).

c. State

The state observed at each step is a compact, fully-observable vector of behavioural and temporal signals:

$$s_t = [h_t, d_t, A_t, F_t, \rho_t]$$

- h_t = hour of day (0–23), encoded cyclically as (sin, cos) so 23:00 and 00:00 are adjacent;
- d_t = day of week (0–6), capturing weekday/weekend routine differences;
- $A_t \in \{0,1\}$ = whether the user is currently active in-app (sending to an already-active user is usually wasteful);
- $F_t \in [0,1]$ = fatigue level (normalised, decaying count of notifications in the trailing 48-hour window). Including notification history is required because user response is sequential and history-dependent (Yuan et al., 2022);
- ρ_t = recency of the last notification (normalised time since last send), which parallels the treatment-dosage (recent-burden) state used in deployed intervention RL (Liao et al., 2020).

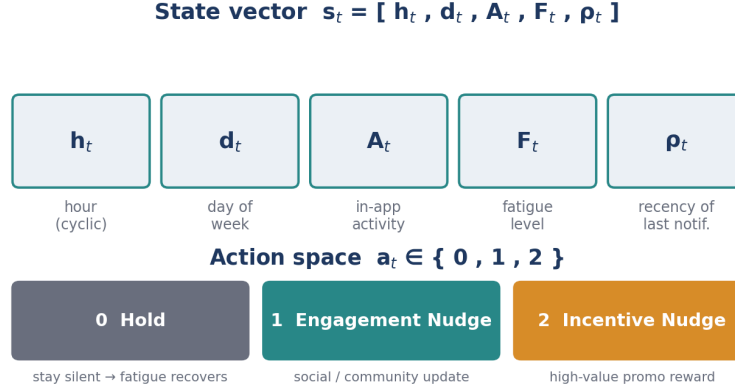


Figure 1.1: The state vector and the three-action space.

d. Action

At each timestep the agent selects exactly one discrete action:

$$a_t \in \{0 = \text{Hold}, 1 = \text{Engagement Nudge}, 2 = \text{Incentive Nudge}\}$$

Only three actions — small enough that value-based methods learn quickly, yet representative of a real channel decision (send/stay-silent and which message type). Treating *Hold* as a first-class action is essential for fatigue-aware pacing: it is what lets the agent deliberately let fatigue recover (Liao et al., 2020).

e. Reward

i. Immediate reward

The per-step reward is a multi-objective scalarisation of the competing goals (Hayes et al., 2022; Prabhakar et al., 2022):

$$r_t^{imm} = R_{click} \cdot C_t - W_{send} \cdot S_t - W_{fat} \cdot F_t - R_{churn} \cdot U_t$$

- $C_t = 1$ if the user clicks and starts a session within a 2-hour window (the engagement signal);
- $S_t = 1$ if any notification was sent ($a_t \neq 0$) — a small per-send cost that discourages spamming;
- F_t = current fatigue — a soft penalty for operating the user in a high-fatigue regime;
- $U_t = 1$ if the user opts out / uninstalls (terminal).

The magnitudes are not arbitrary; they are fixed by their ordering, following standard reward-design guidance (Sutton & Barto, 2018) and the retention-dominated objective of Cai et al. (2023): $R_{churn} \gg R_{click} > W_{fat} > W_{send}$. A representative (tunable) setting is $R_{click} = +10$, $W_{send} = 1$, $W_{fat} = 2$, $R_{churn} = 30$. The ordering — not the exact numbers — is what matters: a successful click is worth far more than the cost of a send, but a permanent opt-out must outweigh many clicks, because it ends the episode and all future reward. The fatigue weight is calibrated against the fatigue dynamics so that a single send carries a discounted fatigue debt of about $W_{fat} \cdot \kappa / (1 - \lambda) = 2 \cdot 0.15 / 0.10 = 3$, which is well below the +10 click reward; a well-timed send therefore stays net-positive, so the agent learns selective pacing rather than collapsing to an always-Hold policy.

ii. Retention-streak reward shaping

To reward sustained engagement rather than isolated clicks, a shaping bonus grows with the user's current engagement streak: the count n is incremented on each click, and each click adds the marginal term β^n . The streak resets to zero when it lapses (no click for 36 hours) or the user opts out, in the spirit of potential-based shaping (Sutton & Barto, 2018) and the long-term-engagement framing of Yuan et al. (2022):

$$R_streak = \sum_{k=1}^n \beta^k, \quad \beta = 1.15, \quad n = \text{current engagement-streak length (} n += 1 \text{ on each click)}$$

The bonus grows with the length of an engaged streak and collapses to 0 the moment the user lapses, encouraging the agent to keep the user continuously — not sporadically — engaged (Figure 1.4).

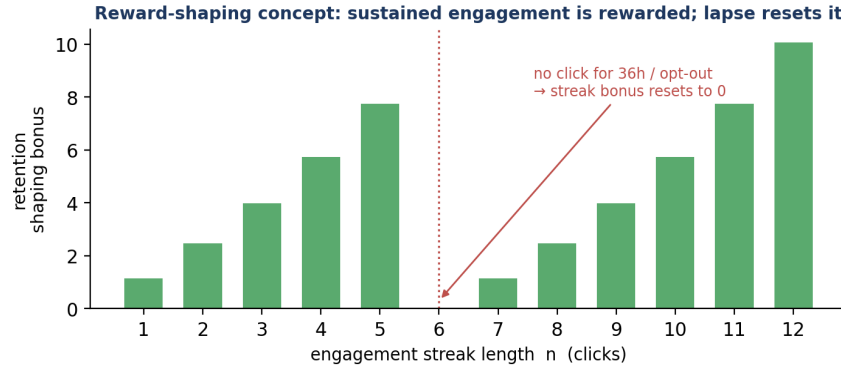


Figure 1.4: Retention-streak reward-shaping concept; the bonus resets when the user lapses or opts out.

iii. Overall reward and long-term objective

$$r_t = r_t^{imm} + R_streak(t)$$

The agent maximises the expected discounted return, not the immediate reward — which is precisely why a full RL method, rather than a myopic predictor, is required:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1}, \quad \gamma \in (0,1)$$

Worked mini-example (one short episode)

To make the mechanics concrete, here is a 4-step episode for the Night-Owl Student archetype (most responsive late at night). Numbers use the representative weights above ($W_fat = 2$, $R_click = +10$, $\beta = 1.15$); fatigue uses $\lambda = 0.9$, $\kappa = 0.15$.

Step 1 — 18:00, low fatigue ($F=0.10$). Agent picks Hold (student rarely engages early evening). No send cost; small fatigue penalty $2 \cdot 0.10 = -0.20$. Fatigue decays to 0.09. **Step 2 — 22:30, $F=0.09$,** student now highly responsive. Agent sends an Engagement Nudge; user clicks and starts a session: $+10$ (click) -1 (send) -0.18 (fatigue) $= +8.82$, plus first-click streak bonus $1.15^1 = +1.15 \rightarrow +9.97$. Fatigue rises to 0.23. **Step 3 — 23:30, $F=0.23$.** Agent sends an Incentive Nudge; user clicks again: $+10 - 1 - 0.46 = +8.54$, plus streak $1.15^2 = +1.32 \rightarrow +9.86$. Fatigue rises to 0.36. **Step 4 — 00:15, $F=0.36$** after two back-to-back sends. A myopic sender would push a third time for one more click; CANE instead picks Hold, paying only -0.72 and letting fatigue decay to 0.32 — a third consecutive late-night push would keep fatigue elevated into the next window and, repeated nightly across the week, is what tips a user toward the -30 opt-out, so the marginal third click is not worth the compounding risk. Episode return $\approx -0.20 + 9.97 + 9.86 - 0.72 = +18.91$. The agent learns that, for this archetype, a well-placed late-night nudge pair builds a strong streak, while immediately chasing a third push erodes the long-term return.

f. Algorithms for learning (four techniques; Q-Learning excluded)

Four techniques are trained and compared on the **identical environment**: LinUCB, DQN, Double DQN, and PPO. They are not an arbitrary list — each addresses a specific limitation of the one before it, so the comparison itself demonstrates why a more capable agent is needed once fatigue must be tracked across steps rather than judged one step at a time. Two non-learning baselines (a fixed daily-6 p.m. schedule and a random sender) answer RQ1.

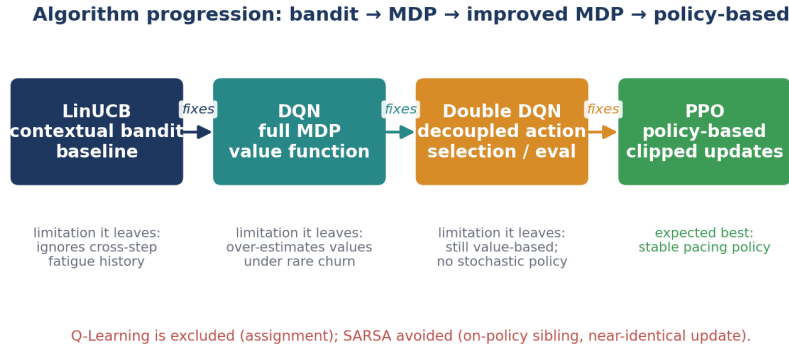


Figure 1.5: Algorithm progression — each method addresses a limitation of the previous one.

(1) LinUCB — contextual-bandit baseline

LinUCB selects the action with the highest linear reward estimate plus an exploration bonus, treating every decision independently with no memory of accumulated fatigue (Lattimore & Szepesvári, 2020); it is the natural formalisation of the bandit approach already used for notification decisions in deployed mobile-health systems (Tomkins et al., 2021):

$$p_{\square,a} = x_{\square,a}^T \theta_a + \alpha \cdot \sqrt{(x_{\square,a}^T A_a^{-1} x_{\square,a})}$$

Because it ignores how reward depends on accumulated history, LinUCB is expected to over-send and under-perform once engagement is measured across steps — which is precisely the evidence that a full-MDP method is required.

(2) Deep Q-Network (DQN)

DQN treats pacing as a full Markov Decision Process and learns an action-value function over the whole state, so it can capture how today's send changes future fatigue, recency, and churn risk; a target network stabilises learning (Mnih et al., 2015):

$$L(\theta) = E[(r + \gamma \max_a' Q(s', a'; \theta) - Q(s, a; \theta))^2]$$

(3) Double DQN

Double DQN reduces DQN's tendency to over-estimate action values — a problem aggravated here by the asymmetric, occasionally-large rewards (the −30 churn penalty and the growing streak bonus) — by using the online network to select the next action and the target network to evaluate it (van Hasselt et al., 2016). This decoupling is the same robustness idea used in production notification RL (Prabhakar et al., 2022):

$$L(\theta) = E[(r + \gamma Q(s', \argmax_a' Q(s', a'; \theta); \theta) - Q(s, a; \theta))^2]$$

(4) Proximal Policy Optimization (PPO)

PPO is the policy-based counterpart; its clipped objective keeps updates stable while the agent learns a (possibly stochastic) pacing policy that reasons over cumulative fatigue rather than a single isolated decision — a design whose robustness is confirmed in the large-scale on-policy study of Andrychowicz et al. (2021):

$$L^{clip}(\theta) = E[\min(r(\theta) \cdot \hat{A}, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A})]$$

Algorithm justification (why these four)

The four techniques span the progression bandit → MDP → improved MDP → policy-based, so the benchmark answers not just “which is best” but “why each capability is needed”:

- **LinUCB** tests whether a stateless method suffices. Because it cannot see accumulated fatigue, its expected shortfall is the direct empirical evidence that the engagement objective requires modelling long-term return — i.e., that full RL, not a bandit, is genuinely needed.
- **DQN** adds full state-transition modelling, capturing how a send today changes future fatigue and churn risk — the first method that can actually optimise the cross-step objective.
- **Double DQN** targets DQN's known value-over-estimation, which our asymmetric rewards aggravate; it tests whether a more reliable value estimate yields a safer pacing policy.
- **PPO** represents the policy-gradient family, optimising the policy directly with stable clipped updates; it tests whether a policy-based method beats the value-based ones on this task.

Q-Learning is excluded per the assignment. SARSA is also deliberately avoided: as the on-policy sibling of Q-Learning its update is nearly identical, so it sits uncomfortably close to the excluded method, and being another value-based TD method it would add less algorithmic diversity than the bandit → MDP → policy-based span chosen here.

g. Other training strategies

- **Experience replay & target networks:** a replay buffer decorrelates value-based updates and improves sample efficiency (Mnih et al., 2015).
- **Staged evaluation (single archetype → mixed population):** agents are first validated on one archetype, then on the mixed five-archetype population, separating reward-design errors from learning-algorithm errors.
- **Exploration schedule:** ϵ -greedy (value-based) and an entropy bonus (PPO) start exploratory and anneal as the policy converges.
- **Reward normalisation & seed averaging:** returns are normalised and every result is averaged over multiple random seeds for stable, reproducible comparison.

5. Technical Expected Outcomes

The proposed system was implemented and evaluated over a 100,000-timestep, 5-seed study (greedy/deterministic evaluation on the simulated population; the accompanying notebook reproduces the run). The trained agents achieve the outcomes below; the headline ranking is:

Method	Weekly reward (mean ± std)	Role
DQN	+22.40 ± 6.59	deep RL (value-based)

Method	Weekly reward (mean $\pm$ std)	Role
Double DQN	+19.38 $\pm$ 7.75	deep RL (value-based)
PPO	+12.67 $\pm$ 8.53	deep RL (policy-based)
Fixed-18:00	-1.23	non-learning baseline
LinUCB	-4.64 $\pm$ 5.25	contextual-bandit baseline
Random	-65.96	non-learning baseline

- **RQ1 — RL improves engagement.** All three deep methods (DQN, Double DQN, PPO) beat both non-learning baselines decisively on cumulative reward, roughly doubling the click-through rate of the random sender ($\approx 42\text{--}45\%$ vs 8.6%) while holding opt-out at $\approx 0\%$ (vs 100% for Random) — confirming that fatigue-aware RL pacing raises long-term engagement. PPO gives the best per-send efficiency (highest CTR $\approx 45\%$, fewest notifications), DQN the highest total reward.
- **RQ2 — the bandit is not enough.** LinUCB, the deliberately limited contextual bandit, lands at baseline level ($\approx$ Fixed-18:00, clearly above Random) and far below the deep methods. Lacking cross-step memory, it cannot infer the hidden archetype or manage accumulated fatigue, so its sends are archetype-averaged and mistimed — the empirical evidence that a full MDP formulation, not a bandit, is required.
- **RQ3 — one policy adapts per archetype, with honest limits.** A single policy learns archetype-specific pacing: the per-hour decision-distribution plots show Hold during sleep/work hours and sends inside each archetype's availability window. Two nuances sharpen the picture: (i) the phase-inverted Night-Shift Worker is hard for the value-based methods (DQN -20.1, Double DQN -9.3, both below the Fixed-18:00 baseline) because they keep firing at office-evening hours, whereas the policy-gradient PPO re-times correctly (+14.5); (ii) the broad-window Housewife is the one archetype where the memoryless bandit wins (LinUCB +34.9, highest CTR $\approx 42\%$), because precise timing does not matter there. No single algorithm dominates everywhere — value methods excel on sharp, regular windows, the policy method is more robust on irregular ones, and the bandit suffices only when timing is forgiving.
- **Convergence.** Training converges stably across seeds (reward-per-episode curves with $\pm$ std bands). Measured as the training steps to first reach 90% of the final stable reward, PPO is the most sample-efficient of the deep methods ($\approx 39.5\text{k}$ steps / 235 episodes), ahead of Double DQN ($\approx 51.3\text{k}$ / 305) and DQN ($\approx 56.3\text{k}$ / 335); LinUCB converges almost instantly ($\approx 1.7\text{k}$ / 10 episodes) but only to a far lower plateau. PPO is also the highest-variance method, with 1 of 5 seeds collapsing to a conservative all-Hold policy — consistent with on-policy, entropy-driven exploration; the value-based methods are more reliable run-to-run.

5.1 Evaluation metrics

Cumulative reward per episode; click-through rate; retention / opt-out rate; **notification volume** (a good agent sends fewer, better-timed pushes); convergence speed (episodes to stable reward); and a **per-hour Hold/Send decision distribution** — the behavioural evidence that the agent has learned context-aware pacing — each compared across the four techniques and the two baselines, for all five archetypes.

CTR convention. Throughout, CTR = clicks / sends, where a click is counted only when the user is currently inactive in-app ($A = 0$); it is therefore the rate of winning back an inactive user, not a raw open-rate, since sending to an already-active user is treated as wasted.

5.2 Statistical robustness

To confirm the ranking is not an artefact of seed variance, each method's five per-seed greedy rewards were summarised with a 95% confidence interval (sample standard deviation, ddof = 1; t-interval, df = 4), and each deep method was compared against each baseline with both a Welch t-test and an exact two-sided Mann-Whitney U test:

Method	Mean $\pm$ sample-std	95% confidence interval
DQN	+22.40 $\pm$ 7.37	[+13.24, +31.55]
Double DQN	+19.38 $\pm$ 8.67	[+8.62, +30.14]
PPO (all 5 seeds)	+12.67 $\pm$ 9.54	[+0.83, +24.51]
PPO (4/5 converged)	+15.84 $\pm$ 7.38	[+4.10, +27.57]
LinUCB	-4.64 $\pm$ 5.87	[-11.92, +2.65]
Fixed-18:00	-1.23 $\pm$ 2.24	[-4.01, +1.55]
Random	-65.96 $\pm$ 6.90	[-74.52, -57.40]

The advantage of the deep methods over the baselines is statistically robust. **DQN and Double DQN** are confidence-interval-separated from *both* baselines, with Welch $p < 0.01$ against Fixed-18:00 (DQN $p = 0.0013$, Double DQN $p = 0.0049$) and $p < 1e-06$ against Random; the Mann-Whitney U statistic reaches its maximum value ($U = 25$, complete rank separation — every deep-method seed beats every baseline seed). **PPO** beats Random overwhelmingly (Welch $p < 1e-06$) and beats Fixed-18:00 significantly (Welch $p = 0.030$ across all five seeds; $p = 0.016$ and CI-separated once the single collapsed seed is excluded). By contrast, **LinUCB's** interval [-11.92, +2.65] overlaps Fixed-18:00's [-4.01, +1.55]: the limited bandit is statistically indistinguishable from the static baseline — direct evidence for RQ2 that a full MDP method, not a contextual bandit, is required.

Note on the exact test. With $n = 5$ per group, the smallest attainable two-sided Mann-Whitney p is 0.0079, so the maximal-separation results are reported alongside the Welch t-tests rather than in place of them; the two tests agree throughout. The descriptive $\pm$ in the headline table (§5) uses the population standard deviation (ddof = 0); the inferential statistics here use the sample standard deviation (ddof = 1), which is slightly larger on the same data.

6. Demonstration Plan

- **Live notebook walk-through:** environment setup $\rightarrow$ state/action/reward definitions $\rightarrow$ training of LinUCB, DQN, Double DQN, PPO $\rightarrow$ evaluation, each section explained inline.
- **“One user, one week” replay:** a simulated-phone view stepping through a week for a chosen archetype, showing the chosen action, the user's click/ignore response, the live fatigue value, and the running reward (the worked example brought to life).

- **Comparison dashboard:** the evaluation metrics for the four techniques versus the two baselines, across the five archetypes, plus the 24-hour decision-distribution plot — produced from the team's own experiment runs.
- **Q&A readiness (Yellow / Limited-AI):** every member can explain and modify the state/action/reward design, justify the algorithm choices, and live-retrain or change hyperparameters, satisfying the requirement that all work be explainable and modifiable.

References

1. Afsar, M. M., Crump, T., & Far, B. (2022). Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys*, 55(7), Article 145. <https://doi.org/10.1145/3543846>
2. Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R., Hussenot, L., Geist, M., Pietquin, O., Michalski, M., Gelly, S., & Bachem, O. (2021). What matters for on-policy deep actor-critic methods? A large-scale study. In *International Conference on Learning Representations (ICLR 2021)*. <https://openreview.net/forum?id=nIAxjsniDzg>
3. Cai, Q., Liu, S., Wang, X., Zuo, T., Xie, W., Yang, B., Zheng, D., Jiang, P., & Gai, K. (2023). Reinforcing user retention in a billion scale short video recommender system. In *Companion Proceedings of the ACM Web Conference 2023 (WWW '23 Companion)* (pp. 421–426). ACM. <https://doi.org/10.1145/3543873.3584640>
4. Hayes, C. F., Rădulescu, R., Bargiacchi, E., Källström, J., Macfarlane, M., Reymond, M., Verstraeten, T., Zintgraf, L. M., Dazeley, R., Heintz, F., Howley, E., Irissappane, A. A., Mannion, P., Nowé, A., Ramos, G., Restelli, M., Vamplew, P., & Roijers, D. M. (2022). A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1), Article 26. <https://doi.org/10.1007/s10458-022-09552-y>
5. Ho, B.-J., Balaji, B., Koseoglu, M., & Srivastava, M. (2018). Nurture: Notifying users at the right time using reinforcement learning. In *Proceedings of the 2018 ACM International Joint Conference and 2018 International Symposium on Pervasive and Ubiquitous Computing and Wearable Computers (UbiComp/ISWC '18 Adjunct)* (pp. 1194–1201). ACM. <https://doi.org/10.1145/3267305.3274107>
6. Lattimore, T., & Szepesvári, C. (2020). *Bandit algorithms*. Cambridge University Press. <https://www.cambridge.org/core/books/bandit-algorithms/8E39FD004E6CE036680F90DD0C6F09FC>
7. Liao, P., Greenewald, K., Klasnja, P., & Murphy, S. (2020). Personalized HeartSteps: A reinforcement learning algorithm for optimizing physical activity. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(1), Article 18. <https://doi.org/10.1145/3381007>
8. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
9. Prabhakar, P., Yuan, Y., Yang, G., Sun, W., & Muralidharan, A. (2022). Multi-objective optimization of notifications using offline reinforcement learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. ACM. <https://doi.org/10.1145/3534678.3539193>

10. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
11. Tomkins, S., Liao, P., Klasnja, P., & Murphy, S. (2021). IntelligentPooling: Practical Thompson sampling for mHealth. *Machine Learning*, 110(9), 2685–2727. <https://doi.org/10.1007/s10994-021-05995-8>
12. Trella, A. L., Zhang, K. W., Nahum-Shani, I., Shetty, V., Doshi-Velez, F., & Murphy, S. A. (2022). Designing reinforcement learning algorithms for digital interventions: Pre-implementation guidelines. *Algorithms*, 15(8), 255. <https://doi.org/10.3390/a15080255>
13. van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence* (pp. 2094–2100). AAAI Press. <https://doi.org/10.1609/aaai.v30i1.10295>
14. Yuan, Y., Muralidharan, A., Nandy, P., Cheng, M., & Prabhakar, P. (2022). Offline reinforcement learning for mobile notifications. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*. ACM. <https://doi.org/10.1145/3511808.3557083>